

Conda Environment Setup for Converting .onnx → .hef

This guide explains how to set up a **conda environment** with the **Hailo Dataflow Compiler (HDC)** so you can compile an `.onnx` model into a `.hef` binary that runs on the **Hailo AI HAT+ (Hailo-10H)** on the Raspberry Pi 5.

It covers all three operating systems:

OS	How conda runs the compiler
Ubuntu / Linux	Native conda env (the supported, tested path)
Windows	conda inside WSL2 (Ubuntu subsystem)
macOS	conda inside a Linux Docker container

🎯 **Target chip:** `hailo10h` . Do **not** compile for `hailo8` / `hailo8l` —

0. Before you start

1. **Have your ONNX model ready** — e.g. `mobilenetv2_cifar10.onnx` from Lab 1 (for testing the environment).
2. **Download the Hailo Dataflow Compiler (.whl)**
 - Create an account at <https://hailo.ai/developer/compilers/> and wait for activation.
 - Once approved, download the **latest Linux .whl** of the compiler, e.g. `hailo_dataflow_compiler-5.3.0-py3-none-linux_x86_64.whl` .
 - The `.whl` is a **Linux** wheel — this is why Windows uses WSL2 and macOS uses Docker.
3. **Install conda** (Miniconda is fine) on your machine.
4. Keep the helper script `graphviz_fix.sh` handy (used if `pygraphviz` fails to build during install).

Throughout this guide, replace `<env_name>` with a name of your choice (e.g. `hailo_convert`) and adjust the `.whl` filename to the version you downloaded.

1. Ubuntu / Linux (native conda)

This is the **recommended and tested** path.

```
# 1. Create and activate a Python 3.10 environment
conda create -n <env_name> python=3.10 -y
conda activate <env_name>
```

```
# 2. Install the Hailo Dataflow Compiler from the .whl you downloaded
pip install hailo_dataflow_compiler-*.whl
```

If the install fails on **graphviz** / **pygraphviz** , install the system libraries first, then re-run the `pip install` :

```
# graphviz_fix.sh
sudo apt update
sudo apt install build-essential -y
sudo apt install graphviz libgraphviz-dev pkg-config -y
pip install pygraphviz
```

Then re-run `pip install hailo_dataflow_compiler-*.whl` .

 Continue to **Section 4 (verify)** and **Section 5 (run the conversion)**.

2. Windows (conda inside WSL2)

The compiler is a Linux wheel, so on Windows we install a lightweight Ubuntu subsystem (WSL2) and run conda inside it.

2.1 Enable WSL2

1. **Settings** → search "Turn Windows features on or off" → tick "Virtual Machine Platform" and "Windows Subsystem for Linux" → OK.
2. Open **Command Prompt** and install WSL (creates an Ubuntu subsystem):

```
wsl --install
```

3. Reboot if prompted, then **open the Ubuntu (WSL) terminal**.

2.2 Locate your `.whl` inside WSL

Your Windows drives are mounted under `/mnt/`. For example, a file on your Desktop is reachable at:

```
cd /mnt/c/Users/<your-windows-user>/Desktop
ls hailo_dataflow_compiler-*.whl
```

2.3 Install Miniconda and create the environment (inside WSL)

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y wget git unzip build-essential
```

```
# Install Miniconda
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
source ~/.bashrc
```

```
# Accept channel terms of service (required by recent conda)
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

```
# Create the Python 3.10 env and install the compiler
conda create -n <env_name> python=3.10 -y
conda activate <env_name>
pip install hailo_dataflow_compiler-*.whl
```

If the install fails on `graphviz`, run the `graphviz_fix.sh` steps from **Section 1**, then re-install the compiler.

 Continue to **Section 4 (verify)** and **Section 5 (run the conversion)** —run them **inside the WSL terminal**.

3. macOS (conda inside a Linux Docker container)

macOS cannot install the Linux `.whl` directly (and Apple Silicon is not x86), so we run everything inside a Linux container.

3.1 Install Docker Desktop

- Download **Docker Desktop for Mac** from the official Docker site (choose **Apple Silicon** for M-series chips, **Intel** otherwise).
- Open the `.dmg` , drag Docker into **Applications**, launch it, and grant permissions.
- Wait until the whale icon shows "**Docker Desktop is running**".

3.2 Start a Linux Python container

```
docker run -it --name hailo_env python:3.10 bash
```

Your prompt changes to something like `root@xxxxxx:/#` — you are now **inside the container**. Keep this window open.

3.3 Copy your files INTO the container

Open a **second** Terminal window (`Cmd + N`) on the Mac and copy the model and the compiler wheel in. Adjust the `~/Desktop/...` paths to where your files are:

```
docker exec hailo_env mkdir -p /workspace
docker cp ~/Desktop/mobilenetv2_cifar10.onnx hailo_env:/workspace/
docker cp ~/Desktop/hailo_dataflow_compiler-*.whl hailo_env:/workspace/
```

3.4 Set up conda and install the compiler (inside the container)

Back in the **first** terminal (inside Docker):

```
cd /workspace
```

```
apt update && apt install -y wget build-essential graphviz libgraphviz-dev pkg-config
```

```
# Install Miniconda
```

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

```
bash Miniconda3-latest-Linux-x86_64.sh
```

```
source ~/.bashrc
```

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
```

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

```
conda create -n <env_name> python=3.10 -y
```

```
conda activate <env_name>
```

```
pip install hailo_dataflow_compiler-*.whl
```

3.5 Copy the result OUT of the container

After the conversion succeeds (Section 5), the `.hef` lives inside the container. From the **second** Mac terminal, copy it back out:

```
docker cp hailo_env:/workspace/mobilenetv2_cifar10.hef ~/Desktop/
```

 Continue to **Section 4 (verify)** and **Section 5 (run the conversion)** —run them **inside the container's first terminal**.

4. Verify the environment

With the environment active, confirm the Hailo client imports cleanly:

```
conda activate <env_name>
```

```
python -c "from hailo_sdk_client import ClientRunner; print('hailo_sdk_client OK')"
```

If this prints `hailo_sdk_client OK`, your environment is ready.

5. Run the conversion (`.onnx` → `.hef`)

✅ Checklist before converting

1. The Jupyter kernel / shell is using the **same conda env** where the HDC is installed.
2. Your `.onnx` file is in the **same folder** as the script/notebook (or pass a full path), otherwise you get a *File not found* error.

5.1 Python

For converting, you can use this python script (adjust the `onnx_path` and `model_name` variables as needed):

```

import os

# Force Hailo's internal TensorFlow onto the CPU.
# The GPU JIT/XLA path often fails in conda envs with mismatched CUDA/cuDNN,
# raising "JIT compilation failed. [Op:Log]" during optimize(). Quantization
# runs fine (just slower) on CPU.
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["TF_XLA_FLAGS"] = "--tf_xla_auto_jit=0"

import numpy as np
from hailo_sdk_client import ClientRunner

# 1. Paths and model name
onnx_path = "mobilenetv2_cifar10.onnx"
model_name = "mobilenetv2_cifar10"
hef_output_path = f"{model_name}.hef"

# 2. Runner for the Hailo-10H chip
runner = ClientRunner(hw_arch="hailo10h")

# 3. Translate ONNX -> Hailo internal format (.har)
print("-> Translating ONNX...")
runner.translate_onnx_model(onnx_path, model_name)

# 4. Calibration dataset for quantization.
# NOTE: PyTorch ONNX is NCHW, but Hailo's parser transposes to NHWC internally,
# so calibration data MUST be NHWC: (N, H, W, C) = (100, 32, 32, 3).
calib_dataset = np.random.rand(100, 32, 32, 3).astype(np.float32)

# 5. Optimize / quantize Float32 -> Int8
print("-> Quantizing (consumes a lot of RAM and time)...")
runner.optimize(calib_dataset)

# 6. Compile to HEF
print("-> Compiling to HEF...")
hef_buffer = runner.compile()

# 7. Save the HEF
with open(hef_output_path, "wb") as f:
    f.write(hef_buffer)

print(f"File saved at: {hef_output_path}")

```

📌 Random calibration data is fine for **FPS / benchmarking**, but for real **accuracy** you should quantize with a representative calibration set captured from your deployment scene.

5.2 ONNX export parameters (avoid conversion errors)

If translation fails, make sure the ONNX was exported from PyTorch with:

```
external_data=False
opset_version=13
export_params=True
dynamo=False          # forces legacy export – avoids newer-PyTorch issues
```

6. Troubleshooting

Symptom	Fix
<code>pip install</code> fails on graphviz / pygraphviz	Run the <code>graphviz_fix.sh</code> steps (Section 1), then re-install the <code>.whl</code> .
JIT compilation failed. [Op:Log] during <code>optimize()</code>	Keep TF on CPU — set <code>CUDA_VISIBLE_DEVICES=-1</code> and <code>TF_XLA_FLAGS=--tf_xla_auto_jit=0</code> before importing <code>hailo_sdk_client</code> (done in the snippet above).
<code>hailo_sdk_client</code> import error / version drift	Recreate the env with Python 3.10 and reinstall the exact <code>.whl</code> .
HEF won't load on device / arch mismatch	You compiled for the wrong target — use <code>hw_arch="hailo10h"</code> .
File not found for the ONNX	Put the <code>.onnx</code> in the same folder as the script/notebook, or pass a full path.
ONNX translate fails on unsupported ops	Re-export with the parameters in Section 5.2 (<code>opset_version=13</code> , <code>dynamo=False</code>).

7. Quick reference

```
# Ubuntu / Linux (native), Windows (inside WSL), macOS (inside Docker):  
conda create -n <env_name> python=3.10 -y  
conda activate <env_name>  
pip install hailo_dataflow_compiler-*.whl  
python -c "from hailo_sdk_client import ClientRunner; print('OK')"  
# then run the Python conversion in Section 5.1
```